Claude Certified Architect Foundations

(CCAR-F)

Practice Exam
Question Set



…by someone who has sat and passed the exam!

Matthew Purcell

linkedin.com/in/purcellmatthew

Claude Certified Architect – Foundations

Practice Question Set

Written following Exam Guide v1.0 (July 2026)

Anthropic's exam guide for the **Claude Certified Architect – Foundations** certification includes twelve sample questions drawn from its practice test — useful, but a fraction of a full sitting. Having sat and passed the exam, I wrote this independent full-length set to add preparation volume: 60 questions matching the live exam's item count, organized into the six published scenarios, distributed across the five content domains in proportion to the official blueprint weights, and pitched at the difficulty level I experienced on exam day.

Be aware that the CCAR-F is scenario-based: the live exam presents four scenarios drawn at random from the published bank of six, with roughly 15 questions framed by each scenario's production context. This set covers all six scenarios, with 10 questions per scenario — so be aware that in the real exam you will be presented with fewer scenarios (four rather than six) with more questions per scenario. Every question is tagged with its blueprint domain so you can score yourself by domain, just like the real score report.

Answers are at the back with concise rationales, so you can attempt each domain cold before checking your reasoning.

Important: these are original practice questions written against the publicly available exam guide and its blueprint objectives. **They are not actual exam content, are not drawn from the live item bank, and do not reproduce any question I encountered.** Exam content is confidential and protected by NDA, as yours will be too. No practice set guarantees a pass; use this alongside hands-on experience and the official documentation.

How the 60 questions are distributed

The blueprint weights from the exam guide, applied to 60 items (each question is also tagged with its domain):

1. Agentic Architecture & Orchestration — **27% → 16 questions**
2. Tool Design & MCP Integration — **18% → 11 questions**
3. Claude Code Configuration & Workflows — **20% → 12 questions**
4. Prompt Engineering & Structured Output — **20% → 12 questions**
5. Context Management & Reliability — **15% → 9 questions**

Question formats you'll encounter

- **Multiple choice.** One correct answer among four options. This is the most common format on the exam. Distractors are plausible — expect to choose between a good answer and the best answer, not between right and obviously wrong.
- **Multiple response.** Select two or more correct answers from the list of options. The question will tell you how many items to select.

Recommended approach: give yourself 120 minutes for all 60 questions — the same time limit as the live exam — and aim for roughly 75% or better in every domain.

Scenario 1: Customer Support Resolution Agent

You are building a customer support resolution agent with the Claude Agent SDK. It handles high-ambiguity requests (returns, billing disputes, account issues) via custom MCP tools — `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human` — targeting 80%+ first-contact resolution while knowing when to escalate.

Question 1.1 · Multiple choice · select ONE · Domain 1

You are implementing the agent's core loop. Which control flow correctly determines when to keep executing tools and when to present the final response?

A. Continue looping until the assistant's text contains a phrase like "I have resolved the issue", then stop

B. Inspect `stop_reason` on each response: while it is "`tool_use`", execute the requested tools and return results for the next iteration; when it is "`end_turn`", present the final response

C. Run exactly five iterations, since support cases rarely need more tool calls than that

D. Stop as soon as any response contains text content, since text indicates completion

Question 1.2 · Multiple choice · select ONE · Domain 1

The agent calls `lookup_order` and your code executes it successfully. How does the model actually receive the order data so it can reason about the next action?

A. The MCP server pushes the result directly into the model's memory

B. The model re-reads the tool's description, which now includes the result

C. Nothing is required — the API executes tools and incorporates results automatically

D. Your code appends a tool result block (referencing the tool call's ID) to the conversation history and sends the updated messages back to Claude in the next request

Question 1.3 · Multiple choice · select ONE · Domain 1

Company policy caps autonomous refunds at $500; larger refunds must go to a human. In testing, prompt instructions alone still let an occasional $650 refund through. What is the correct enforcement design?

A. Implement a hook that intercepts outgoing `process_refund` calls, blocks any amount above $500, and redirects the agent to the `escalate_to_human` workflow

B. Move the $500 limit to the top of the system prompt and repeat it in every user turn

C. Lower the model temperature so it follows the policy more consistently

D. Add the limit to the `process_refund` tool description so the model reads it during selection

Question 1.4 · Multiple choice · select ONE · Domain 1

A customer writes one message containing three concerns: a refund for a damaged item, a shipping address change, and a missing loyalty credit. How should the agent handle this?

A. Resolve only the refund, since it was mentioned first, and wait for the customer to raise the others again

B. Escalate immediately — multi-issue messages always exceed agent capability

C. Decompose the message into three distinct items, investigate each (in parallel where possible) using the shared customer context, then synthesize one unified resolution covering all three

D. Ask the customer to resend each issue as a separate message

Question 1.5 · Multiple choice · select ONE · Domain 1

The agent must escalate a complex billing dispute mid-process. The human agents receiving escalations do not have access to the conversation transcript. What should the escalation include?

A. Only the customer's name, so the human can start fresh without bias

B. A structured handoff summary: customer ID, the root cause analysis so far, relevant amounts and order numbers, and the agent's recommended action

C. A sentiment score describing how upset the customer is

D. The raw JSON of every tool result gathered during the session

Question 1.6 · Multiple choice · select ONE · Domain 2

When `process_refund` fails for any reason, it currently returns the string “Operation failed.” The agent responds by retrying policy-blocked refunds and giving up on temporary outages — the opposite of correct. What is the fix?

A. Instruct the agent in the system prompt to always retry exactly twice

B. Remove failure responses entirely so the agent never sees errors

C. Have the agent apologize to the customer whenever any error occurs

D. Return structured error metadata: an `errorCategory` (transient / validation / business / permission), an `isRetryable` boolean, and a human-readable description the agent can act on

Question 1.7 · Multiple choice · select ONE · Domain 2

A refund request violates the returns window policy. The tool rejects it. Which error response enables the agent to handle this correctly with the customer?

A. `errorCategory`: “business”, `retriable: false`, plus a customer-friendly explanation of the policy — so the agent explains the refusal rather than retrying it

B. `errorCategory`: “transient” so the agent retries later, when the policy might differ

C. An empty result marked as success, to keep the conversation positive

D. A stack trace, so the agent has maximum detail

Question 1.8 · Multiple response · select TWO · Domain 2

The agent frequently confuses `get_customer` and `lookup_order`; both currently carry one-line descriptions. Which TWO description changes most directly fix the misrouting? (Select TWO.)

A. Make both descriptions shorter so there is less to confuse

B. Document each tool's expected input formats and add example queries it should handle

C. Prefix both descriptions with “IMPORTANT:” for emphasis

D. Add boundary language differentiating the two — e.g., “use `lookup_order` when an order number is present; use `get_customer` to verify identity before account operations”

E. Rely on the tool names alone, since names are what models read

Question 1.9 · Multiple choice · select ONE · Domain 5

A customer's first message is: “I want to speak to a human. Do not try to fix this yourself.” The issue looks like a routine replacement the agent could resolve. What should the agent do?

A. Attempt resolution first, escalating only if the attempt fails

B. Run sentiment analysis and escalate only if frustration exceeds a threshold

C. Escalate immediately — an explicit customer request for a human is honored without first attempting investigation

D. Ask the customer three clarifying questions before deciding

Question 1.10 · Multiple response · select TWO · Domain 5

In long multi-issue sessions, the agent's later responses misquote refund amounts and order numbers because earlier details were condensed into vague summaries, while each `lookup_order` result dumps 40+ fields into context. Which TWO changes address this? (Select TWO.)

A. Extract transactional facts — amounts, dates, order numbers, statuses — into a persistent case-facts block included in each prompt, outside the summarized history

B. Increase `max_tokens` so responses can be longer

C. Summarize even more aggressively to save space

D. Trim each tool result to only the fields relevant to the current issue before it enters context

E. Ask the customer to repeat key numbers periodically

Scenario 2: Code Generation with Claude Code

Your team uses Claude Code for code generation, refactoring, debugging, and documentation, integrated into daily development with custom slash commands and `CLAUDE.md` configuration — and you must judge when plan mode beats direct execution.

Question 2.1 · Multiple choice · select ONE · Domain 3

You have a personal `/scratch-notes` slash command you use constantly, but it reflects your own workflow and should not appear for teammates when they pull the repository. Where does the command file belong?

A. In `.claude/commands/` in the repository, like all slash commands

B. In the root `CLAUDE.md` under a # Commands heading

C. In `~/.claude/commands/` in your home directory, where user-scoped commands live outside version control

D. In `.claude/rules/` with a paths glob matching your username

Question 2.2 · Multiple choice · select ONE · Domain 3

Your `/analyze-architecture` skill produces thousands of lines of exploration output that pollute the main conversation, crowding out the task you actually wanted help with. Which `SKILL.md` frontmatter option fixes this?

A. `argument-hint`, so the skill asks what to analyze

B. `context: fork`, which runs the skill in an isolated sub-agent context and returns results without flooding the main conversation

C. `allowed-tools`, restricting the skill to read-only operations

D. paths, restricting the skill to architecture files

Question 2.3 · Multiple choice · select ONE · Domain 3

You are writing a `/scaffold-component` skill that generates boilerplate files. You want a guarantee it can create files but can never run shell commands, no matter what its instructions are asked to do. How do you enforce this?

A. State “never run `Bash`” prominently in the `SKILL.md` instructions

B. Trust the skill's narrow purpose to keep it away from `Bash`

C. Run the skill only in plan mode

D. Set `allowed-tools` in the `SKILL.md` frontmatter to permit only the file-write operations the skill needs, excluding `Bash`

Question 2.4 · Multiple choice · select ONE · Domain 3

A new teammate's Claude Code sessions ignore the team's coding standards. You discover the standards live in your own `~/.claude/CLAUDE.md`. What is the correct fix, and how do you verify it?

A. Move the standards into the project-level `CLAUDE.md` (checked into the repository) so every clone receives them, and use the `/memory` command to verify which memory files a session has loaded

B. Email the teammate your `~/.claude/CLAUDE.md` to copy into their home directory

C. Have the teammate re-install Claude Code to refresh configuration

D. Paste the standards at the start of each session until they stick

Question 2.5 · Multiple choice · select ONE · Domain 3

The project's `CLAUDE.md` has grown to 2,000 lines covering testing, API conventions, deployment, and styling; sessions load it all regardless of task. What is the recommended reorganization?

A. Delete the file and rely on the model's general knowledge of good practice

B. Move everything to each developer's user-level `CLAUDE.md`

C. Split it into focused, topic-specific files in `.claude/rules/` (e.g., `testing.md`, `api-conventions.md`, `deployment.md`), adding path-scoped frontmatter where conventions apply only to matching files

D. Convert the entire file into a single slash command developers run manually

Question 2.6 · Multiple choice · select ONE · Domain 1

Yesterday you ran a long investigation into a memory leak in a session you named `leak-hunt`. Today you want to continue exactly where that investigation left off. What do you run?

A. `claude --continue-all`, which reopens every recent session

B. `claude --resume leak-hunt`, resuming the named session with its context

C. `claude -p` "continue the leak investigation"

D. A new session, pasting yesterday's terminal scrollback as the first message

Question 2.7 · Multiple choice · select ONE · Domain 1

You have completed a thorough analysis of a module and now want to compare two competing refactoring strategies — developing each independently from that same analysis baseline without the explorations contaminating each other. Which capability fits?

A. Run both strategies sequentially in the same session, asking Claude to forget the first before starting the second

B. Open two brand-new sessions and redo the module analysis in each

C. Use `/compact` between the two strategies

D. Use `fork_session` to create independent branches from the shared analysis baseline, exploring one strategy per branch

Question 2.8 · Multiple choice · select ONE · Domain 1

You want to resume last week's refactoring session, but since then the team has merged substantial changes across the files it analyzed, so most of its tool results now describe code that no longer exists. What is the reliable approach?

A. Start a new session seeded with a structured summary of the prior conclusions — stale tool results make resumption unreliable, since the model will reason from outdated file contents

B. Resume the session as-is; Claude automatically detects file changes

C. Resume the session and increase `max_tokens` to accommodate the confusion

D. Never resume any session; always start from zero exploration

Question 2.9 · Multiple choice · select ONE · Domain 5

Mid-way through an extended exploration session, context is filling with verbose discovery output and responses are noticeably degrading, but you are not ready to end the session. Which built-in command helps immediately?

A. `/memory`, which clears the conversation

B. `/fork`, which splits the session in half

C. `/compact`, which reduces context usage by compacting the conversation so the session can continue productively

D. `/rules`, which reloads the rule files

Question 2.10 · Multiple response · select TWO · Domain 5

During multi-hour codebase exploration sessions, the agent starts answering from “typical patterns” instead of the specific classes it discovered earlier. Which TWO practices counteract this context degradation? (Select TWO.)

A. Switch to a model with a larger context window and continue accumulating

B. Maintain a scratchpad file recording key findings as they are discovered, and have the agent reference it for subsequent questions

C. Repeat important questions twice in the same message

D. Ask the agent to be more specific

E. Delegate verbose investigation (e.g., “find all test files”, “trace the refund flow”) to subagents that return summaries, keeping the main session's context for high-level coordination

Scenario 3: Multi-Agent Research System

A coordinator agent built on the Claude Agent SDK delegates to specialized subagents — web search, document analysis, synthesis, and report generation — to research topics and produce comprehensive, cited reports.

Question 3.1 · Multiple choice · select ONE · Domain 1

Your coordinator's system prompt tells it to delegate research to subagents, but at runtime it never spawns any — it attempts all the research itself. Its configuration restricts it to research-planning tools. What is the most likely cause?

A. The coordinator model tier is too small to understand delegation

B. Subagents must be started manually before the coordinator runs

C. The system prompt needs the word “delegate” in capital letters

D. The coordinator's `allowedTools` does not include “`Task`” — the `Task` tool is the mechanism for spawning subagents, so without it no delegation can occur

Question 3.2 · Multiple choice · select ONE · Domain 1

The web search subagent gathers excellent findings, but the synthesis subagent's reports are generic and reference none of them. The coordinator invokes synthesis with the prompt “Synthesize the research findings.” What is wrong?

A. The synthesis agent's model needs extended thinking enabled

B. Subagents do not inherit the coordinator's conversation history — the coordinator must include the actual findings (search results, analysis outputs) directly in the synthesis subagent's prompt

C. The synthesis agent needs web search tools to gather its own findings

D. The prompt needs the word “please”

Question 3.3 · Multiple choice · select ONE · Domain 1

The coordinator currently invokes the web search subagent, waits for completion, then invokes the document analysis subagent — doubling research latency even though the two tasks are independent. How do you run them in parallel?

A. Have the coordinator emit both `Task` tool calls in a single response, which spawns the subagents in parallel

B. Ask each subagent to work faster in its system prompt

C. Run two copies of the entire system and merge the databases

D. Parallelism is impossible — subagents must always run sequentially

Question 3.4 · Multiple choice · select ONE · Domain 1

Your coordinator gives subagents rigid step-by-step procedures (“run these exact five queries in this order”). Subagents fail whenever a topic doesn't fit the script. How should coordinator prompts be designed instead?

A. Make the procedures longer, covering more contingencies explicitly

B. Remove all instructions and let subagents infer their tasks

C. Specify research goals and quality criteria — what a complete answer looks like — and let the subagent adapt its approach to what it discovers

D. Have subagents ask the human operator whenever the script fails

Question 3.5 · Multiple choice · select ONE · Domain 1

Completed reports are coherent but shallow on some subtopics. You want the system to notice and repair its own coverage gaps before finalizing. Which orchestration pattern achieves this?

A. Lower the bar: accept the first synthesis as final to control costs

B. An iterative refinement loop: the coordinator evaluates synthesis output for gaps, re-delegates to search and analysis subagents with targeted queries, and re-invokes synthesis until coverage is sufficient

C. Ask the report generation agent to pad thin sections with general knowledge

D. Always run every subagent exactly twice regardless of output quality

Question 3.6 · Multiple choice · select ONE · Domain 2

Every subagent currently receives the full 18-tool catalog. Logs show the synthesis agent attempting web searches and the search agent trying document parsing — with frequent wrong-tool selections everywhere. What is the correct redesign?

A. Add a nineteenth tool that tells the agents which tool to use

B. Alphabetize the tool list so selection is easier

C. `Write` longer system prompts warning each agent about the tools it should ignore

D. Scope each subagent's tool set to its role — a handful of relevant tools each — since large tool inventories degrade selection reliability and out-of-role tools invite misuse

Question 3.7 · Multiple choice · select ONE · Domain 2

Subagents burn many tool calls just discovering what data exists — listing available document collections, probing for issue summaries, checking what schemas are present — before real work begins. Which MCP capability reduces this?

A. Expose content catalogs (document hierarchies, issue summaries, schema listings) as MCP resources, giving agents visibility into available data without exploratory tool calls

B. Increase the tool-call budget so exploration is affordable

C. Hard-code the current data inventory into every system prompt

D. Remove discovery capability so agents work blind

Question 3.8 · Multiple choice · select ONE · Domain 5

Final reports state findings but cite nothing. Investigation shows each summarization step compresses source attribution away, so by synthesis time nobody knows which claim came from where. What is the structural fix?

A. Have the report generator add plausible citations at the end

B. Instruct the synthesis agent to write “sources available on request”

C. Require subagents to output structured claim-source mappings (claim, evidence excerpt, source URL or document name) that every downstream agent preserves and merges through synthesis

D. Reduce the number of summarization steps to one

Question 3.9 · Multiple response · select TWO · Domain 5

Two credible sources report different figures for the same market's size, and one subagent's summary simply picked the larger number. Which TWO practices produce correct handling of conflicting source data? (Select TWO.)

A. Annotate the conflict explicitly — both values, each with source attribution — and let the coordinator decide how to reconcile before synthesis, rather than silently selecting one

B. Average the two figures into a single compromise value

C. Always adopt the more recent source and discard the other

D. Require publication or data-collection dates in subagents' structured outputs, so temporal differences are not misread as contradictions

E. Exclude both sources to avoid taking sides

Question 3.10 · Multiple response · select TWO · Domain 5

You are designing how the web search subagent behaves when its searches fail or return nothing. Which TWO behaviors are correct? (Select TWO.)

A. On any failure, immediately terminate the entire research workflow

B. Attempt local recovery for transient failures (e.g., retrying a timeout), and propagate to the coordinator only errors it cannot resolve — including what was attempted and any partial results

C. Return empty results marked as success whenever a search fails, keeping the pipeline moving

D. Report every problem as a generic “search unavailable” status

E. Distinguish access failures (timeouts, service errors) from valid empty results (successful queries with no matches), since the two require different coordinator decisions

Scenario 4: Developer Productivity with Claude

You are building developer productivity tooling on the Claude Agent SDK: helping engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate, and automate repetitive tasks — using the built-in tools (`Read`, `Write`, `Edit`, `Bash`, `Grep`, `Glob`) plus MCP servers.

Question 4.1 · Multiple response · select TWO · Domain 2

The agent must (a) locate every file matching the test naming convention anywhere in the repo, and (b) find every place the string "`PaymentDeclinedError`" appears in code. Which TWO tool selections are correct? (Select TWO.)

A. `Glob` with a pattern like `**/*.test.tsx` for the test files — file-path pattern matching is `Glob`'s job

B. `Grep` with a filename pattern to find the test files

C. `Grep` for "`PaymentDeclinedError`" — searching file contents for a string is `Grep`'s job

D. `Glob` for "`PaymentDeclinedError`" across file contents

E. `Bash` running `cat` on every file, piping through manual inspection

Question 4.2 · Multiple choice · select ONE · Domain 2

The agent attempts an `Edit` on a legacy file and fails: the anchor text it targeted appears in six places, so the unique-match requirement cannot be satisfied. What is the standard fallback?

A. Delete five of the six occurrences so the match becomes unique

B. Retry the same `Edit` repeatedly until it succeeds

C. Switch to `Bash` and modify the file with `sed`

D. Use `Read` to load the full file, then `Write` the complete modified content — the reliable path when `Edit` cannot find unique anchor text

Question 4.3 · Multiple choice · select ONE · Domain 2

Asked how the billing module works, the agent Reads dozens of files upfront and exhausts its context before answering. What is the correct exploration strategy for building codebase understanding?

A. `Read` files in alphabetical order and stop at the context limit

B. Work incrementally: `Grep` to find entry points (key functions, imports, error strings), then `Read` selectively to follow imports and trace flows from those anchors

C. Ask the developer to explain the module instead

D. `Read` only the README and reason from documentation

Question 4.4 · Multiple choice · select ONE · Domain 2

The team shares a GitHub MCP server that needs an auth token, and you also run a personal, experimental MCP server you don't want to impose on anyone. How should these be configured?

A. The shared server in project-scoped `.mcp.json` using environment variable expansion (e.g., `${GITHUB_TOKEN}`) so no secret is committed; the personal server in user-scoped `~/.claude.json`

B. Both servers in `.mcp.json`, with the token pasted in so teammates don't have to set variables

C. Both servers in `~/.claude.json`, with setup instructions in the wiki

D. The shared server in `CLAUDE.md` and the personal one in `.claude/commands/`

Question 4.5 · Multiple choice · select ONE · Domain 3

Two tasks arrive: fixing an off-by-one bug in one function with a clear stack trace, and migrating the codebase's HTTP library — touching 45+ files with several viable approaches. How should plan mode and direct execution be assigned?

A. Plan mode for both — planning never hurts

B. Direct execution for both — plan mode is only for greenfield projects

C. Direct execution for the well-scoped single-file bug fix; plan mode for the migration, whose scale and multiple valid approaches demand exploration and design before changes are committed

D. Plan mode for the bug fix and direct execution for the migration

Question 4.6 · Multiple choice · select ONE · Domain 3

Developers keep invoking your `/generate-fixture` skill bare, without the entity type and record count it needs, then get confused by the results. Which frontmatter option addresses this?

A. `context: fork`, isolating the confusion in a subagent

B. `allowed-tools`, restricting what the skill can touch

C. A longer skill description explaining the parameters

D. `argument-hint`, which prompts developers for the required parameters when the skill is invoked without arguments

Question 4.7 · Multiple choice · select ONE · Domain 3

You must decide where two things live: (1) the team's universal coding standards that should shape every session, and (2) a multi-step release-notes workflow used a few times per month. What is the correct placement?

A. Both as skills, invoked when developers remember

B. Standards in `CLAUDE.md`, which is always loaded; the occasional release-notes workflow as a skill in `.claude/skills/`, invoked on demand

C. Both in `CLAUDE.md` so nothing is ever missed

D. Standards as a skill; the release workflow in `CLAUDE.md`

Question 4.8 · Multiple choice · select ONE · Domain 1

The task is open-ended: “add comprehensive tests to this legacy codebase.” Nobody knows yet where the risk concentrates or what depends on what. Which decomposition approach fits?

A. First map the codebase structure, identify high-impact areas, then create a prioritized plan that adapts as dependencies and surprises are discovered

B. `Write` tests file-by-file in directory order until done

C. Generate all tests in a single maximal request

D. Test only the code modified in the last month

Question 4.9 · Multiple choice · select ONE · Domain 1

You are choosing decomposition strategies for two workflows: a code review that always checks the same five aspects, and a production-incident investigation whose next step depends on what each finding reveals. Which pairing is right?

A. Dynamic decomposition for both, since flexibility is always better

B. Prompt chaining for both, since fixed steps are easier to test

C. Prompt chaining (a fixed sequence of focused passes) for the predictable five-aspect review; dynamic adaptive decomposition for the investigation, where intermediate findings determine the next subtask

D. Dynamic decomposition for the review; prompt chaining for the investigation

Question 4.10 · Multiple response · select TWO · Domain 1

You are writing `AgentDefinition` configurations for the productivity system's subagents. Which TWO statements about subagents are accurate? (Select TWO.)

A. Subagents automatically inherit the parent agent's full conversation history

B. Each subagent's `AgentDefinition` carries its own description, system prompt, and tool restrictions for its role

C. Subagents can be spawned without the `Task` tool if the prompt describes them clearly

D. Subagents do not share memory between invocations — whatever context a subagent needs must be provided explicitly in its prompt

E. Subagent tool restrictions are advisory and not enforced at runtime

Scenario 5: Claude Code for Continuous Integration

You are integrating Claude Code into a CI/CD pipeline for automated code review, test generation, and pull-request feedback — designing prompts that produce actionable findings while minimizing false positives.

Question 5.1 · Multiple choice · select ONE · Domain 3

Your CI job must run Claude Code non-interactively and produce review findings your pipeline can parse and post as inline PR comments. Which invocation is correct?

A. claude “review this PR” piped through grep to extract findings from prose

B. `claude -p` “review this PR” with `--output-format json` and `--json-schema`, producing machine-parseable structured findings without waiting for interactive input

C. claude `--batch` “review this PR” `--format=structured`

D. `claude -p` “review this PR” alone, then regex-parsing the prose output

Question 5.2 · Multiple choice · select ONE · Domain 3

CI-generated tests ignore your team's fixture library, duplicate helper setup, and test trivialities. Developers reject most of them. What is the configuration-level fix?

A. Generate three times as many tests so some survive review

B. Have developers rewrite the generated tests as a standing chore

C. Lower the temperature of the generation call

D. Document testing standards, what makes a test valuable, and the available fixtures in `CLAUDE.md` — the mechanism that supplies project context to CI-invoked Claude Code

Question 5.3 · Multiple choice · select ONE · Domain 3

Reviews re-run after each new commit to a PR, and the bot re-posts the same findings every time — developers now mute it. How should re-reviews be designed?

A. Include the prior review's findings in context and instruct Claude to report only new issues or previously reported ones that remain unaddressed

B. Review only the newest commit's diff in isolation

C. Limit reviews to one per PR regardless of subsequent commits

D. Post all findings again but formatted differently to seem new

Question 5.4 · Multiple choice · select ONE · Domain 3

You notice that when the same Claude Code session that generated a change also reviews it, the review is conspicuously gentle — missing issues an independent reviewer catches. Why, and what is the design implication?

A. The model is being polite; instruct it to be harsher

B. Reviews should always be performed by a larger model tier

C. A session retains the reasoning context from generation, making it less likely to question its own decisions — so CI reviews should run in an independent instance without the generator's context

D. Generation and review must run on different days to avoid contamination

Question 5.5 · Multiple choice · select ONE · Domain 4

Your review prompt says “be conservative and only report high-confidence findings,” yet false positives remain high. What does effective precision engineering look like instead?

A. Add “be very, very conservative” for stronger emphasis

B. Report all findings but sort them by the model's stated confidence

C. Reduce the number of files reviewed per run

D. Replace confidence language with explicit categorical criteria: define which issue types to report (bugs, security) and which to skip (minor style, local patterns), with concrete boundaries

Question 5.6 · Multiple choice · select ONE · Domain 4

Findings in the “code style” category are 70% false positives, and developers have begun dismissing security findings too — trust in the whole reviewer is collapsing. What is the right operational move?

A. Ship more style findings to demonstrate the category's importance

B. Temporarily disable the style category to restore trust in the accurate categories, while improving the style prompts before re-enabling it

C. Rename “code style” to “code quality”

D. Ask developers to be more tolerant of noise during the tuning period

Question 5.7 · Multiple choice · select ONE · Domain 4

The reviewer labels near-identical issues “critical” one day and “minor” the next. Severity-based merge gates are therefore unreliable. How do you get consistent severity classification?

A. Define explicit severity criteria with concrete code examples for each level, so every severity has an operational definition the model can match against

B. Remove severity levels and treat all findings equally

C. Let the model choose severity freely but average it over five runs

D. Map severity to line count of the affected code

Question 5.8 · Multiple choice · select ONE · Domain 4

The reviewer keeps flagging your codebase's accepted idiomatic patterns (e.g., intentional fall-throughs with comments) as bugs, while your detailed prose instructions haven't fixed it. What is the most effective addition?

A. A rule that anything containing a comment is acceptable

B. An instruction to “use good judgment about idioms”

C. Few-shot examples distinguishing the accepted patterns from genuine issues — showing the reasoning why each is or isn't reportable — so the model generalizes the judgment to novel cases

D. A ban on reviewing files containing any idiomatic pattern

Question 5.9 · Multiple response · select TWO · Domain 4

Your team is deciding which CI workloads to move to the Message Batches API for its 50% cost savings. Which TWO statements are accurate? (Select TWO.)

A. Batch processing suits latency-tolerant, non-blocking workloads like nightly test generation and weekly audit reports

B. Batches are guaranteed to complete within one hour

C. The batch API supports multi-turn tool calling within a single request

D. Blocking workflows such as pre-merge checks should stay on the synchronous API, since batches can take up to 24 hours with no latency SLA

E. Batch responses cannot be matched back to their requests

Question 5.10 · Multiple response · select TWO · Domain 4

You are adding few-shot examples to the review prompt. Which TWO practices reflect how few-shot prompting works best? (Select TWO.)

A. Include as many examples as possible — twenty or more — to cover every case

B. Use 2–4 targeted examples aimed at the ambiguous scenarios, showing the reasoning for why one action was chosen over plausible alternatives

C. Once examples are added, explicit criteria become unnecessary

D. Rely on examples only for prose tasks; they don't work for code

E. Include examples demonstrating the exact desired output format — location, issue, severity, suggested fix — to achieve consistent, actionable findings

Scenario 6: Structured Data Extraction

You are building a structured data extraction system: pulling information from unstructured documents, validating output against JSON schemas, handling edge cases gracefully, and integrating with downstream systems.

Question 6.1 · Multiple choice · select ONE · Domain 4

Your current pipeline asks Claude to “respond with JSON” in plain text; downstream parsing breaks on markdown fences, trailing commentary, and occasional malformed syntax. What is the most reliable structural fix?

A. Strengthen the prompt: “respond ONLY with valid JSON, no exceptions”

B. Post-process the text with regexes that strip fences and repair syntax

C. Define the extraction as a tool whose input schema is your JSON schema, and read the structured data from the `tool_use` block — guaranteeing schema-compliant output and eliminating JSON syntax errors

D. Switch to XML output, which models format more carefully

Question 6.2 · Multiple choice · select ONE · Domain 4

You have several extraction tools — one per document type — and incoming documents of unknown type. The model must always produce a structured extraction, choosing the appropriate schema itself, and never reply with conversational text. Which `tool_choice` setting is correct?

A. `tool_choice`: “any” — the model must call a tool but may choose which, guaranteeing structured output while leaving schema selection to the model

B. `tool_choice`: “auto” — the model decides whether to call a tool at all

C. `tool_choice` forced to one named tool, applied to every document

D. Omit `tool_choice`, since defaults handle this

Question 6.3 · Multiple choice · select ONE · Domain 4

Your invoice schema marks `vendor_tax_id` as required. On invoices that genuinely lack a tax ID, the model fabricates plausible-looking values to satisfy the schema. What is the schema-level fix?

A. Add a prompt instruction: “never fabricate tax IDs”

B. Post-validate tax IDs against a checksum and discard failures

C. Lower the temperature to make fabrication less creative

D. Make the field optional/nullable so the model can legitimately return null when the information is absent from the source document

Question 6.4 · Multiple choice · select ONE · Domain 4

Since adopting tool use, extractions are always syntactically valid — yet invoices arrive where line items don't sum to the stated total, and values occasionally land in the wrong fields. What should you understand and do?

A. Schema compliance was falsely advertised; file a bug

B. Strict schemas eliminate syntax errors but not semantic ones — add semantic validation, e.g., extracting `calculated_total` alongside `stated_total` and flagging discrepancies with a `conflict_detected` boolean

C. Make every field an unconstrained string so nothing can be wrong

D. Re-run each extraction until the numbers happen to sum

Question 6.5 · Multiple choice · select ONE · Domain 4

An extraction fails Pydantic validation with two specific field errors. You will retry. What should the retry request contain — and when would you skip retrying altogether?

A. Just the instruction “try again more carefully”; skip retries on Mondays

B. Only the validation errors, to keep the retry cheap; always retry at least five times

C. The original document, the failed extraction, and the specific validation errors — enabling targeted self-correction; skip retrying when the required information is simply absent from the source, since no retry can produce it

D. A different document, to give the model a fresh start

Question 6.6 · Multiple choice · select ONE · Domain 4

An overnight batch of 10,000 documents completes with 200 failures: most are oversized documents that blew past context limits, plus some transient errors. What is the correct failure-handling workflow?

A. Identify the failed requests by `custom_id` and resubmit only those, with modifications where needed — e.g., chunking the oversized documents before resubmission

B. Resubmit the entire 10,000-document batch and keep whichever results arrive first

C. Discard the 200 failures as an acceptable loss rate

D. Switch the 200 failures to the synchronous API unchanged, where context limits don't apply

Question 6.7 · Multiple choice · select ONE · Domain 2

A single `analyze_document` tool handles extraction, summarization, and claim verification, chosen by a mode parameter. The agent regularly picks the wrong mode and results are inconsistent. What is the recommended redesign?

A. Add a fourth mode that automatically detects the right mode

B. Document the modes more thoroughly inside the one description

C. Reduce to two modes by merging extraction into summarization

D. Split the generic tool into purpose-specific tools — `extract_data_points`, `summarize_content`, `verify_claim_against_source` — each with a defined input/output contract

Question 6.8 · Multiple choice · select ONE · Domain 2

When your MCP extraction tool hits a parsing failure, it returns the message “Error: could not parse document” as an ordinary successful text result. The agent then treats that sentence as document content. What is the correct MCP pattern?

A. Prefix error text with “SYSTEM ERROR:” so the agent notices

B. Return the failure with the MCP `isError` flag set, so the agent programmatically recognizes a tool failure instead of interpreting error prose as data

C. Return an empty string on failure

D. Throw an unhandled exception and let the connection drop

Question 6.9 · Multiple response · select TWO · Domain 5

Your extraction system reports 97% aggregate accuracy, and leadership wants to cut human review of high-confidence extractions. Which TWO practices must precede that decision? (Select TWO.)

A. Segment accuracy by document type and field — verifying consistent performance across all segments, since aggregate metrics can mask systematic failures on specific types

B. Accept the 97% aggregate as sufficient evidence on its own

C. Implement stratified random sampling of high-confidence extractions for ongoing error-rate measurement and detection of novel error patterns

D. Stop measuring once review is reduced, to save cost

E. Review only extractions the model itself flags as uncertain

Question 6.10 · Multiple response · select TWO · Domain 5

Reviewer capacity covers only a fraction of extractions, so review attention must be routed where errors are likeliest. Which TWO practices form a sound routing design? (Select TWO.)

A. Trust the model's raw self-reported confidence scores without validation

B. Have the model output field-level confidence scores, then calibrate review thresholds against a labeled validation set

C. Review a fixed 1% of extractions chosen by submission order

D. Route to human review the extractions with low calibrated confidence or ambiguous/contradictory source documents, prioritizing limited reviewer capacity where errors concentrate

E. Review only documents longer than ten pages

Answer Key & Rationales

Detailed reasoning for each correct answer, with a brief hint on why the remaining options fall short.

Scenario 1: Customer Support Resolution Agent

1.1 — Correct: B

The agentic loop keys off `stop_reason`: "`tool_use`" means the model has requested tools and expects their results back; "`end_turn`" means it has finished reasoning and produced its answer. That signal is the designed termination mechanism.

Why not the others: *A parses natural language for a completion signal — a documented anti-pattern; C makes an arbitrary iteration cap the primary stopping mechanism; D fails because responses can contain both text and `tool_use` blocks mid-task.*

1.2 — Correct: D

Tool execution is the harness's job: the loop appends the tool result to the conversation and re-invokes the model. That growing history is the only channel through which the model sees what tools returned.

Why not the others: *A and C describe automatic mechanisms that do not exist — the API does not execute your tools; B confuses static tool definitions with runtime results.*

1.3 — Correct: A

Business rules with financial consequences need deterministic guarantees. A tool-call interception hook enforces the threshold in code — the violating call can never execute — while prompt-based approaches remain probabilistic.

Why not the others: *B and D improve the odds but retain a non-zero failure rate, which the scenario shows is unacceptable here; C misapplies a sampling parameter to a compliance problem.*

1.4 — Correct: C

Multi-concern decomposition is the designed pattern: split the request into distinct items, investigate each against shared context, and respond once with a complete resolution — protecting first-contact resolution.

Why not the others: *A and D silently drop or bounce concerns the customer already stated, directly harming FCR; B escalates work that is well within capability.*

1.5 — Correct: B

Structured handoff protocols exist precisely because the receiving human lacks the transcript: the summary transfers the case state — identity, findings, figures, and a recommendation — so the customer never has to start over.

Why not the others: *A discards completed investigation and forces repetition; C conveys mood, not case state; D transfers unprocessed noise and leaves the analysis to be redone.*

1.6 — Correct: D

Uniform error strings deny the agent the information recovery decisions require. Structured metadata — category, retryability, description — lets it retry transient failures, explain business refusals, and stop wasting attempts on non-retryable errors.

Why not the others: *A hard-codes one recovery strategy for error types that need different ones; B removes the signal instead of enriching it; C treats communication as a substitute for correct recovery logic.*

1.7 — Correct: A

Policy violations are business errors: non-retryable by definition, and best paired with an explanation the agent can relay. The `retriable: false` flag prevents wasted attempts; the friendly description powers the customer conversation.

Why not the others: *B mislabels a deterministic refusal as temporary, guaranteeing futile retries; C is silent error suppression — an anti-pattern that turns failure into misinformation; D exposes internals the agent cannot use and should not repeat.*

1.8 — Correct: B, D

Descriptions are the primary selection mechanism. Input formats plus example queries show each tool's territory; explicit boundary language resolves precisely the overlap causing the misrouting.

Why not the others: *A removes information from tools that already have too little; C adds emphasis without discrimination; E is false — names help, but descriptions carry the selection signal.*

1.9 — Correct: C

Explicit customer demands for a human are a first-class escalation trigger, honored immediately. Overriding a stated preference to demonstrate capability damages trust regardless of how resolvable the issue looks.

Why not the others: *A and D do exactly what the customer forbade; B substitutes an unreliable proxy (sentiment) for a signal that could not be clearer — the customer already said the words.*

1.10 — Correct: A, D

The paired fix: protect precise transactional facts from lossy summarization by persisting them in a dedicated block, and stop the bloat at its source by trimming verbose tool outputs to relevant fields before they accumulate.

Why not the others: *B controls output length, not context quality; C intensifies the exact failure mode — numbers dissolving into vagueness; E outsources the system's memory problem to the customer.*

Scenario 2: Code Generation with Claude Code

2.1 — Correct: C

Command scoping mirrors intent: `~/.claude/commands/` holds personal commands visible only to you, while `.claude/commands/` in the repo is shared with everyone via version control. A personal workflow tool belongs in the former.

Why not the others: *A publishes a personal tool to the whole team; B is project context, not a command definition mechanism; D misuses path-scoped rules, which condition on edited file paths, not identity.*

2.2 — Correct: B

`context: fork` exists for exactly this: verbose or exploratory skills run in their own isolated context, and the main session receives the distilled result rather than the full exploration transcript.

Why not the others: *A governs parameter prompting; C governs which tools the skill may use — neither controls where output accumulates; D is rules-file syntax, not skill frontmatter for isolation.*

2.3 — Correct: D

`allowed-tools` is the enforcement mechanism: frontmatter-declared tool restrictions bound what the skill can invoke during execution — a configuration guarantee rather than an instruction the model might not follow.

Why not the others: *A is prompt-level guidance with a non-zero failure rate; B is hope; C changes the execution workflow but is not a per-skill tool restriction.*

2.4 — Correct: A

User-level configuration applies only to that user — it never travels via version control. Team standards belong at project level, and `/memory` is the diagnostic that shows exactly which memory files a session loaded.

Why not the others: *B distributes configuration by hand and drifts immediately; C reinstalls software to fix a file-location problem; D mistakes a persistent configuration issue for a per-session prompting task — and nothing "sticks" between sessions.*

2.5 — Correct: C

The `.claude/rules/` directory is the modular alternative to a monolithic `CLAUDE.md`: focused topic files, with YAML path scoping where rules should load only for matching files — less irrelevant context, easier maintenance.

Why not the others: *A abandons hard-won project conventions; B unshares team configuration and guarantees divergence; D makes always-relevant standards opt-in and forgettable.*

2.6 — Correct: B

Named session resumption is the mechanism for continuing specific prior work: `--resume <session-name>` restores that conversation's context so the investigation picks up where it stopped.

Why not the others: *A is not a real flag; C starts a fresh non-interactive run with no memory of yesterday; D approximates resumption badly — unstructured scrollback is a poor substitute for the session's actual context.*

2.7 — Correct: D

`fork_session` exists for divergent exploration: both branches inherit the completed analysis, then evolve independently — clean comparison without re-paying the analysis cost or cross-contaminating approaches.

Why not the others: *A is impossible — context cannot be selectively forgotten on request; B pays for the expensive baseline twice; C compresses context within one session, it does not branch it.*

2.8 — Correct: A

The resumption tradeoff: resume when prior context is mostly valid; start fresh with an injected summary when tool results have gone stale. Heavily changed files put this squarely in the second case — keep the conclusions, discard the outdated evidence.

Why not the others: *B assumes an automatic re-verification that does not happen — stale results sit in context as if true; C spends tokens on a correctness problem; D throws away a valid technique — for minor changes, resuming and informing the session which files changed works well.*

2.9 — Correct: C

`/compact` is the in-session relief valve for exactly this situation: it compacts accumulated context — typically bloated with verbose discovery output — so an extended session can continue without degradation.

Why not the others: *A shows which memory files are loaded — diagnostic, not cleanup; B is not the mechanism (forking is for divergent branches, not compression); D does not address conversation bloat.*

2.10 — Correct: B, E

Both practices manage what occupies the context: a scratchpad persists precise findings outside the degrading conversation, and subagent delegation keeps verbose exploration out of the main window entirely, returning only distilled summaries.

Why not the others: *A postpones the same degradation at higher cost; C and D add more text to an already saturated context without removing any of the noise causing the problem.*

Scenario 3: Multi-Agent Research System

3.1 — Correct: D

Subagent spawning happens through the `Task` tool. A coordinator whose `allowedTools` omits "`Task`" has no mechanism to delegate, no matter how clearly its prompt describes the intention — capability configuration trumps instruction.

Why not the others: *A and C look for the failure in comprehension or emphasis when the tool inventory makes delegation impossible; B describes a startup model that does not exist — subagents are spawned on demand.*

3.2 — Correct: B

Subagent context isolation is the rule: each subagent sees only what its prompt contains. “The research findings” refers to material the synthesis agent has never seen — the coordinator must pass the findings explicitly.

Why not the others: *A adds reasoning capacity over an empty input; C duplicates the search agent's role instead of fixing the handoff; D is not the missing ingredient — data is.*

3.3 — Correct: A

Parallel spawning is achieved by emitting multiple `Task` tool calls in one coordinator response rather than across separate turns — independent workstreams then execute concurrently.

Why not the others: *B changes nothing about scheduling; C is massive infrastructure duplication for a one-line orchestration fix; D is false — parallel `Task` calls are a documented pattern.*

3.4 — Correct: C

Goal-and-criteria prompts preserve subagent adaptability: the coordinator defines success, the subagent chooses the path — which is why the agent (not a fixed script) is there at all.

Why not the others: *A is an arms race against topic variety that scripts always lose; B removes the goal along with the script; D reintroduces a human into a loop designed to run autonomously.*

3.5 — Correct: B

Iterative refinement closes the quality loop: evaluate coverage, dispatch targeted follow-up research where gaps exist, re-synthesize — repeating until the report meets criteria rather than hoping the first pass suffices.

Why not the others: *A ships the known defect; C fills gaps with uncited generalities — the opposite of a cited research product; D doubles cost blindly with no gap-detection to aim the second pass.*

3.6 — Correct: D

Tool distribution is an architectural control: agents choose more reliably among 4–5 role-relevant tools than among 18, and tools outside an agent's specialization are misused precisely because they are available.

Why not the others: *A adds selection complexity to solve selection complexity; B changes ordering, not decision difficulty; C asks prompts to fight an inventory problem configuration should fix.*

3.7 — Correct: A

MCP resources exist for exactly this: exposing catalogs of available content so agents start informed. Discovery becomes a lookup instead of a spelunking expedition of exploratory calls.

Why not the others: *B pays for the inefficiency rather than removing it; C goes stale the moment the data changes; D trades wasted calls for ignorance.*

3.8 — Correct: C

Provenance survives only if it is structural: claim-source mappings travel as data through every hop, so synthesis merges attributed claims instead of anonymous assertions. Attribution is preserved, never reconstructed.

Why not the others: *A invents citations — worse than none in a research product; B advertises attribution the system no longer possesses; D reduces compression events but the remaining one still strips attribution.*

3.9 — Correct: A, D

Conflicts are information: preserve both values with attribution and surface the disagreement for deliberate reconciliation — and carry dates in structured output, since “conflicting” figures often just measure different moments.

Why not the others: *B invents a number no source reported; C automates a reconciliation that needs judgment — recency is context, not an override rule; E deletes two credible sources because they disagree.*

3.10 — Correct: B, E

Resilient error propagation is layered and honest: handle locally what is locally fixable, escalate the rest with context the coordinator can act on, and never conflate “the search broke” with “the search found nothing.”

Why not the others: *A makes any single failure fatal when partial-result strategies exist; C is silent suppression — failure dressed as fact; D hides the very context that recovery decisions need.*

Scenario 4: Developer Productivity with Claude

4.1 — Correct: A, C

The division of labor is clean: `Glob` matches file paths against patterns (names, extensions, directories); `Grep` searches file contents for patterns (identifiers, error strings, imports). Each task maps to exactly one of them.

Why not the others: *B and D swap the tools into each other's territory; E abandons both purpose-built tools for a brute-force pipeline that floods context.*

4.2 — Correct: D

`Edit` depends on unique text matching; when a file's repetitive structure defeats that, `Read` + `Write` performs the modification reliably on the whole file. It is the documented fallback for exactly this failure.

Why not the others: *A mutilates code to satisfy the tool; B retries a deterministic failure — the sixfold match will not resolve itself; C swaps a controlled file operation for fragile stream editing.*

4.3 — Correct: B

Incremental exploration is the pattern: search first to find where the relevant code lives, then read narrowly along the traced paths. Context is spent on the files that matter rather than on everything alphabetically prior to them.

Why not the others: *A spends the budget by filename accident; C outsources the agent's job to its user; D trusts documentation to describe code accurately — in a legacy system, a famous last assumption.*

4.4 — Correct: A

Scoping follows audience: project `.mcp.json` distributes team tooling via version control, with ${VAR} expansion keeping credentials out of the repo; `~/.claude.json` holds personal and experimental servers that stay yours.

Why not the others: *B commits a live secret to version control; C makes shared infrastructure a manual wiki chore that drifts; D puts server configuration in files that don't configure servers.*

4.5 — Correct: C

The assignment rule: direct execution for simple, well-scoped changes; plan mode where scale, architectural implications, or competing approaches make design-before-commitment valuable — like a 45-file migration.

Why not the others: *A taxes a trivial fix with ceremony; B invites costly rework by executing a large migration unplanned; D inverts the rule on both tasks.*

4.6 — Correct: D

`argument-hint` is the frontmatter mechanism for exactly this: skills that need parameters can prompt for them on bare invocation instead of running underspecified.

Why not the others: *A isolates output, not input requirements; B constrains tools, not arguments; C documents the parameters for whoever reads docs — the invocation still runs bare.*

4.7 — Correct: B

The dividing line: `CLAUDE.md` for universal, always-relevant standards; skills for on-demand, task-specific workflows. Each mechanism carries the load it was designed for.

Why not the others: *A makes universal standards opt-in — they will be skipped; C loads a monthly workflow into every session's context forever; D inverts both placements.*

4.8 — Correct: A

Open-ended tasks call for adaptive decomposition: understand the terrain, prioritize by impact, and let the plan evolve with what each step reveals — the opposite of a fixed pipeline chosen before anything is known.

Why not the others: *B spends effort by directory-listing order rather than risk; C asks one pass to do what requires discovery; D uses recency as a proxy for importance — legacy risk often lives in old, untouched code.*

4.9 — Correct: C

The selection rule: fixed sequential pipelines for predictable multi-aspect work, adaptive decomposition where the path emerges from intermediate findings. Each workflow gets the structure its uncertainty profile demands.

Why not the others: *A pays adaptivity overhead on a task with no uncertainty; B forces an investigation to follow steps written before the evidence existed; D assigns each strategy to the workflow that defeats it.*

4.10 — Correct: B, D

Two load-bearing facts: subagents are configured individually (description, system prompt, tool restrictions), and they are isolated — no inherited history, no memory across invocations, so context passing is always explicit.

Why not the others: *A and C invert the isolation and spawning models — context never flows automatically, and `Task` is the spawning mechanism; E is false — tool restrictions are configuration, not suggestion.*

Scenario 5: Claude Code for Continuous Integration

5.1 — Correct: B

The CI trio: `-p` (`--print`) for non-interactive execution, `--output-format json` for machine-readable output, and `--json-schema` to enforce the findings structure your pipeline consumes — no prose parsing, no input hangs.

Why not the others: *A hangs waiting for interactive input and then scrapes prose; C invents flags that do not exist; D solves the hang but leaves the pipeline regex-parsing unstructured text.*

5.2 — Correct: D

CI invocations get their project context from `CLAUDE.md`. Standards, value criteria, and fixture documentation there directly raise generation quality — the model can only follow conventions it has been shown.

Why not the others: *A scales the reject pile; B institutionalizes rework instead of fixing its cause; C reduces variability of output that is uninformed either way.*

5.3 — Correct: A

Duplicate suppression is a context design problem: give the reviewer its own prior findings and the explicit instruction to report deltas — new issues plus unresolved carryovers — and the noise stops while coverage remains complete.

Why not the others: *B misses issues that emerge from interaction with earlier commits; C leaves everything after the first push unreviewed; D disguises the spam developers already muted.*

5.4 — Correct: C

Session context isolation matters for review integrity: the generating session carries the rationale that produced the code, biasing it toward its own choices. An independent instance evaluates the code on its own terms.

Why not the others: *A misreads a context effect as a personality setting; B changes capability when the problem is contaminated context — the same model reviews well when independent; D adds superstition to a mechanism that is about context, not calendars.*

5.5 — Correct: D

Vague conservatism doesn't transfer — specific criteria do. Defining reportable versus skippable categories with concrete boundaries gives the model an operable decision rule, which is what actually moves precision.

Why not the others: *A intensifies an instruction that has no operational content; B reorders noise instead of reducing it — self-stated confidence is poorly calibrated; C reviews less code with the same faulty judgment.*

5.6 — Correct: B

High false-positive categories are contagious: they teach developers to dismiss everything. Disabling the offender protects the credibility of accurate categories while its prompts are fixed offline — trust is the system's real asset.

Why not the others: *A doubles down on the noise destroying trust; C relabels the same false positives; D asks humans to absorb a cost the configuration should eliminate.*

5.7 — Correct: A

Consistency comes from operational definitions: severity levels anchored by concrete examples give the classifier something to match, turning a vibe into a rubric. That is what makes severity gates dependable.

Why not the others: *B discards the signal the merge gate needs; C statistically launders an unanchored judgment; D measures size, not impact — a one-line auth bypass outranks a fifty-line comment tweak.*

5.8 — Correct: C

When prose fails to convey a judgment boundary, examples carry it: contrasted acceptable-versus-genuine cases with reasoning teach a distinction the model can generalize — the documented strength of few-shot prompting for false-positive reduction.

Why not the others: *A creates a trivially wrong rule any commented bug defeats; B restates the goal without transferring the judgment; D exempts exactly the code that most needs reviewing.*

5.9 — Correct: A, D

The batch decision rule in both directions: overnight and weekly jobs are the ideal profile for the discount; anything a developer waits on cannot tolerate a 24-hour, no-SLA window and stays synchronous.

Why not the others: *B and C are false — there is no latency guarantee, and mid-request tool execution is unsupported; E is false — `custom_id` exists precisely to correlate request/response pairs.*

5.10 — Correct: B, E

Effective few-shot work is targeted and demonstrative: a handful of examples aimed at genuine ambiguity, with reasoning shown, plus format demonstrations that lock output structure — quality of targeting over quantity.

Why not the others: *A bloats context and dilutes the signal of the examples that matter; C and D are false — examples complement explicit criteria, and code review is a prime few-shot use case.*

Scenario 6: Structured Data Extraction

6.1 — Correct: C

Tool use with JSON schemas is the reliability mechanism for structured output: the model's extraction arrives as schema-conformant tool input, not as prose that happens to contain JSON — syntax errors are eliminated by construction.

Why not the others: *A improves the odds within a fundamentally text-shaped channel; B patches symptoms with fragile repairs; D relocates the same free-text problem to a different syntax.*

6.2 — Correct: A

The three modes map to intent: “any” compels a tool call while preserving the model's choice among schemas — exactly right for unknown document types that must always yield structured output.

Why not the others: *B (and D, which defaults to it) permits conversational text instead of a tool call; C welds every document to one schema when types vary.*

6.3 — Correct: D

Required fields on possibly-absent information force fabrication — the schema leaves no honest answer. Nullable/optional fields give the model a legitimate way to say “not present,” which is the designed prevention for this failure.

Why not the others: *A pits an instruction against a structural requirement that demands a value; B catches well-formed fabrications only by luck; C makes invented values more conservative-looking, not less invented.*

6.4 — Correct: B

The boundary of the guarantee: tool use ensures structure, not truth. Semantic errors — sums that don't reconcile, transposed fields — need a semantic validation layer, with self-check fields like `calculated_total` making discrepancies machine-visible.

Why not the others: *A misunderstands what was promised: shape, not semantics; C destroys the structure that downstream systems depend on; D retries until errors align by chance rather than detecting them.*

6.5 — Correct: C

Retry-with-error-feedback works because the model sees what it produced, what was wrong, and the source to correct against. And the boundary matters: format and structural errors are retryable; information absent from the document is not.

Why not the others: *A retries blind — the model can't fix errors it isn't shown; B removes the document and extraction the correction must reference; D abandons the failed extraction rather than repairing it.*

6.6 — Correct: A

`custom_id` exists for exactly this: correlate failures to their source documents, fix what caused each failure (chunk the oversized ones), and resubmit only the fixed subset — paying again for 200 documents, not 10,000.

Why not the others: *B reprocesses 9,800 successes to retry 200 failures; C silently drops 2% of the corpus; D misunderstands context limits, which belong to the model, not the API path — unchunked oversized documents fail synchronously too.*

6.7 — Correct: D

Purpose-specific tools turn a hidden mode decision into a visible selection decision — the thing tool descriptions are good at guiding. Each tool's contract is explicit, and misrouting drops accordingly.

Why not the others: *A buries the selection problem one layer deeper; B improves documentation of a structure that remains ambiguous; C reduces options while keeping the overloaded design.*

6.8 — Correct: B

`isError` is MCP's channel for communicating tool failure: it makes the error a machine-recognizable condition the agent can reason about, rather than text masquerading as a successful result.

Why not the others: *A still relies on the model inferring failure from prose conventions; C replaces a mislabeled failure with a silent one; D turns a recoverable tool error into a transport-level breakdown.*

6.9 — Correct: A, C

Two safeguards make automation defensible: segmentation proves the average isn't hiding a failing document type or field, and stratified sampling of the “safe” population keeps measuring after the humans step back — catching drift and novel errors.

Why not the others: *B trusts exactly the number that masks segment failures; D removes the instrumentation precisely when risk increases; E trusts self-flagged uncertainty, which misses the errors the model is confidently wrong about.*

6.10 — Correct: B, D

Calibration then routing: field-level confidence scores become meaningful once thresholds are tuned on labeled data, and review capacity flows to the calibrated-low-confidence and ambiguous-source cases where errors actually cluster.

Why not the others: *A uses uncalibrated confidence — the known-unreliable version of the right signal; C and E route by arrival order and page count, neither of which correlates with error risk.*

How did you go? If this helped your CCAR-F preparation, share it with someone else who's preparing.

For more, follow linkedin.com/in/purcellmatthew